



uv Package Manager -- Get Your Sunscreen!

uv Package Manager -- Get Your Sunscreen!

Date: May 13, 2026

Presenter: Mohal Khandelwal

Website: <http://www.colorado.edu/rc>

Documentation: <https://curc.readthedocs.io>



Slides

https://github.com/ResearchComputing/rmacc_2026/

Meet the User Support Team



Layla
Freeborn



Brandon
Reyes



Andy
Monaghan



Michael
Schneider



John
Reiland



Dylan
Gottlieb



Mohal
Khandelwal



Ragan
Lee

What is uv?

- Fast, modern Python package manager and environment builder.
- Built by Astral (same team behind Ruff).
- Implemented in Rust.
- All-in-one solution
- Designed to work natively with Python's built-in venv



Image source: <https://docs.astral.sh/uv/>

Features of uv

- Drop-in Replacement for pip, pip-tools.
- 10-100x faster than traditional tools.
- Reliable and reproducible builds
- Memory-efficient operation, especially for large projects

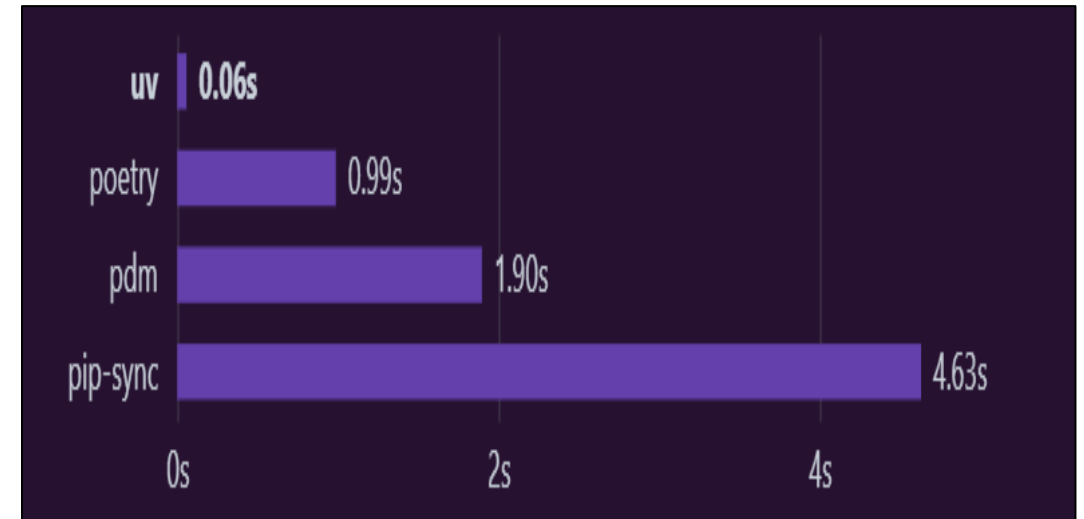


Image source: <https://docs.astral.sh/uv/>

uv vs. the status quo

Feature	uv	Traditional pip + venv	conda
Speed	10-100x faster than pip	Baseline	Slower than pip
Memory Usage	Very efficient	Higher	High
Reproducibility	Universal lockfile (uv sync).	Requires a separate tool (pip-tools) for locking.	YAML files (often flaky).
Non-Python Packages	No	No	Yes
Dependency Resolution	Fast, modern resolver	Basic	Comprehensive
Ease of use	Simple, single tool	Broader scope with a steeper learning curve.	Requires multiple commands (python -m venv, source, pip).



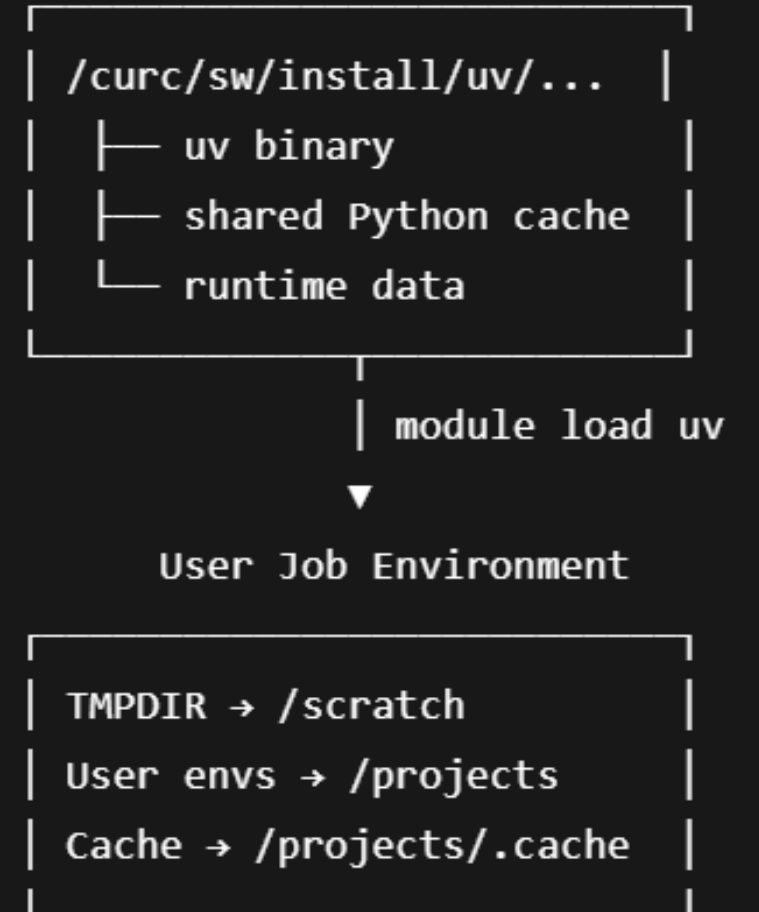
uv Installation

- Installed using the upstream Astral bootstrap installer

```
$ curl -LsSf  
https://astral.sh/uv/install.sh  
| sh
```

- Loading the module:
 - adds the uv executable to `PATH`
 - redirects temporary files to scratch storage
 - configures per-user directories under `/projects/\$USER`

Shared Software Stack



uv Installation

- Loading the module sets the following environment variables specific to the user:
 - **UV_CACHE_DIR**: Package and wheel cache
 - **UV_DATA_DIR**: uv runtime data
 - **UV_PYTHON_INSTALL_DIR**: Managed Python installs
 - **UV_ENVS**: User virtual environments
 - **TMPDIR**: User's scratch directory

```
/projects/$USER/software/uv/  
├─ bin/           → user-installed executables  
├─ envs/          → uv-managed virtual environments  
├─ cache/         → package and wheel cache  
├─ config/        → uv configuration files  
├─ data/          → uv runtime data  
│   └─ python/    → uv-managed Python installations  
└─ tools/         → uv tool installations
```


Getting A CURC Account

- CU Boulder, CSU users and affiliates:
 - Request an account through the RC Account request portal:
<https://rcamp.rc.colorado.edu/accounts/account-request/create/organization>
- AMC, RMACC users and affiliates:
 - Request an account through the ACCESS-CI User Registration Portal:
<https://identity.access-ci.org/new-user.html>

Getting Started with uv on CURC

- Start an interactive job from a CURC login node:

```
$ acompile
```

- Then load the module in your interactive session:

```
$ module spider uv
```

```
$ module load uv
```

```
$ uv --version
```

Creating a uv Environment

- uv automates the creation and management of environments using **venv**
- **venv** is Python's built-in tool for creating isolated environments
- Each **venv**:
 - Has its own Python interpreter
 - Has its own installed packages, separate from system Python
 - Avoids dependency conflicts across projects
- It's a lightweight alternative to Conda environments

Creating a uv Environment

- Loading the module sets a key environment variable **\$UV_ENVS**.

```
$ echo $UV_ENVS  
/projects/$USER/software/uv/envs
```

- To create a new environment

```
$ uv venv $UV_ENVS/myenv  
$ uv venv $UV_ENVS/myenv1 --python 3.8
```


Managing Python versions

- You can create virtual environments with specific Python versions using the `--python` flag.

```
$ uv venv $UV_ENVS/mycustomenv --python 3.8
```

Note: While uv can be used with beta versions of Python, this is not recommended, as many community-contributed modules may not function correctly until an official Python release is available.

Activating the Environment

- Packages are installed within activated environments
 - Fully compatible with PyPI
 - Uses caching to speed up repeated installs

```
$ source $UV_ENVS/myenv/bin/activate
```

```
$ uv pip install numpy
```

```
$ uv pip install pandas
```

Useful Commands

<code>uv <command> --help</code>	<code># help for specific commands</code>
<code>uv venv \$UV_ENVS/<envname></code>	<code># create a new environment</code>
<code>uv pip install <packagename></code>	<code># install a package in active env</code>
<code>uv pip uninstall <packagename></code>	<code># uninstall a package from env</code>
<code>rm -rf \$UV_ENVS/<envname></code>	<code># remove an environment</code>
<code>uv cache clean</code>	<code># clean up unused cache</code>
<code>uv pip freeze</code>	<code># list installed packages</code>
<code>deactivate</code>	<code># deactivate environment</code>

Note: Be very careful with `rm -rf` when removing environments

uv Performance

- Virtual environment creation

```
$ time python -m venv .venv
```

```
real    0m29.315s
```

```
user    0m1.991s
```

```
sys     0m0.483s
```

```
$ time uv venv $UV_ENVS/mycustomenv
```

```
Using CPython 3.13.7
```

```
Creating virtual environment at: /projects/mokh8410/software/uv/envs/mycustomenv
```

```
Activate with: source /projects/mokh8410/software/uv/envs/mycustomenv/bin/activate
```

```
real    0m4.938s
```

```
user    0m0.046s
```

```
sys     0m0.040s
```



Installing packages (cold cache)

- With no cached wheels, installing five libraries (boto3, requests, pandas, fastapi, sqlalchemy) and their 23 total dependencies:

```
$ time pip install boto3 requests pandas fastapi sqlalchemy
```

```
...
```

```
Successfully installed 23 packages
```

```
5.65s user 1.19s system 13.137 total
```

```
$ time uv pip install boto3 requests pandas fastapi sqlalchemy
```

```
...
```

```
Installed 23 packages in 58ms
```

```
0.30s user 0.63s system 0.871 total
```

- 15x faster, and the gap widens with more packages.

Installing packages (warm cache)

- With packages already cached:

```
$ time pip install boto3 requests pandas fastapi sqlalchemy
...
Successfully installed 23 packages
4.91s user 0.96s system 6.597 total

$ time uv pip install boto3 requests pandas fastapi sqlalchemy
...
Installed 23 packages in 91ms
0.03s user 0.11s system 0.150 total
```

- Reuses cached pre-built wheels and hard links files into environments, making installs nearly instantaneous.

Summary

Note: Results will vary by hardware, network speed, and package size.

Operation	pip	uv	Speedup
Create virtual environment	29.3s	4.9s	6x
Install 23 packages (cold cache)	13.1s	0.87s	15x
Install 23 packages (warm cache)	6.6s	0.15s	44x

More Documentation!



Official uv Docs!

<https://docs.astral.sh/uv/>

CURC uv Guide

<https://curc.readthedocs.io/en/latest/software/uv.html>



Questions



Comments